

Máquinas Universais, Turing e λ -Cálculo

Projeto Final — Teoria da Computabilidade (AV2)

Modelos: λ -Cálculo $\rightarrow$ Combinadores SKI · MT Não Determinística

Turma CC5NA — Prof. Daniel Leal Souza — 01/2026

Carlos Eduardo Cardoso Silva · Gabriel Costa de Miranda · João Ricardo Silva de Almeida

Agenda

1. **Teoria** — o que é computar e a Hipótese de Church-Turing
2. **Modelo 1** — λ -Cálculo $\rightarrow$ Combinadores **SKI** (Python)
3. **Modelo 2** — Máquina de Turing **Não Determinística** (JFLAP)
4. **Análise transversal** — determinismo $\times$ não determinismo, λ  combinadores
5. **Encerramento** + arguição

Dois **problemas distintos**: transformação de termos $\times$ reconhecimento de linguagem.

Teoria — o que é "computar"?

- Vários formalismos tentaram capturar a noção intuitiva de **algoritmo**:
Máquina de Turing, λ -cálculo, combinadores, funções recursivas, Máquina de Post...
- **Hipótese de Church-Turing**: todos eles definem a **mesma** classe de funções computáveis. Não importa o modelo — o que é computável é computável em qualquer um.
- Nosso projeto mostra isso na prática com **dois modelos bem diferentes** que, ainda assim, são equivalentes em poder à Máquina de Turing.

Modelo 1 — λ -Cálculo $\rightarrow$ SKI: o problema

Computar no λ -cálculo é fazer β -redução: $(\lambda x.E) M \rightarrow E[x := M]$.

Implementar a substituição exige **gerenciar nomes**:

- variáveis **livres** $\times$ **ligadas**;
- evitar **captura** de variáveis (α -conversão).

Pergunta: dá pra computar **sem variáveis nenhuma**?

SKI — três combinadores

Combinador	Regra	Papel
I	$I\ x \rightarrow x$	identidade
K	$K\ x\ y \rightarrow x$	guarda o 1º, descarta o 2º
S	$S\ x\ y\ z \rightarrow x\ z\ (y\ z)$	distribui o argumento

E o **I** é dispensável: $S\ K\ K\ x \rightarrow^* x$.

$S\ K\ K\ x == [S] ==> K\ x\ (K\ x) == [K] ==> x$

Bastam **S** e **K** — sistema Turing-completo sem nomes.

SKI — eliminação de abstração

Algoritmo $T[\lambda x. E]$ traduz λ -termo $\rightarrow$ SKI puro:

- $T[\lambda x. x] = I$
- $T[\lambda x. E] = K E$ (se x não ocorre em E)
- $T[\lambda x. (E F)] = S (T[\lambda x. E]) (T[\lambda x. F])$

$T[\lambda x. x]$	$T[\lambda x. y]$	$T[\lambda x. (x y)]$
I	$K y$	$S I (K y)$

A burocracia de nomes some na tradução; o motor de execução é só casamento de padrão.

SKI — demonstração (traço de 7 passos)

Numeral de Church 2 = `S (S (K S) K) (S K K)` → "aplicar `f` duas vezes":

```
S (S (K S) K) (S K K) f x ==[S]
S (K S) K f (S K K f) x   ==[S]
K S f (K f) (S K K f) x   ==[K]
S (K f) (S K K f) x       ==[S]
K f x (S K K f x)         ==[K]
f (S K K f x)             ==[S]
f (K f (K f) x)           ==[K]
=> f (f x)
```

`python implementacoes/lambda-ski/demo.py` · `Ω = S I I (S I I)` não normaliza

(Problema da Parada).

Modelo 2 — MTND: o problema

Reconhecer números compostos em unário:

$$L = \{ 1^n : n = a \cdot b, a \geq 2, b \geq 2 \}$$

- Entrada: n em unário ($6 = 111111$).
- **Aceita** se composto; **rejeita** se primo, $\emptyset$ ou 1 .

1111 (4)	111111 (6)	11111 (5)	1
✓	✓	✗ primo	✗

MTND — por que **não** determinística?

"Existe um fator a de n com quociente ≥ 2 ?"

- Perguntas do tipo "**existe...**" são a motivação clássica do não determinismo.
- A máquina **adivinha** o fator a — cada palpite é um **ramo** da árvore.
- **Aceita se algum ramo** encontra a fatoraçoão.

Único ponto não determinístico: q_2 lendo $1 \rightarrow$

$\delta(q_2, 1) = \{(q_2, A, R), (q_3, 1, L)\}$ (continuar marcando **ou** parar).

MTND — como funciona (duas fases)

Fase 1 — adivinhar a (não determinística)

- Marca os primeiros 1 s como A (gabarito do divisor), forçando $a \geq 2$.

Fase 2 — verificar (determinística)

- Cruza os 1 s restantes em blocos de tamanho a (cada $1 \rightarrow B$).
- Falta 1 no meio do bloco $\rightarrow$ o ramo **morre**.
- Zerou os 1 s exatamente $\rightarrow$ **aceita** (garante $b \geq 2$).

12 estados · 24 transições (acima do mínimo da lauda).

MTND — árvore de computação 1^6

```
raiz: q0 sobre 111111
├── a=2 → ACEITA (6 = 2×3)
├── a=3 → ACEITA (6 = 3×2)
├── a=4 → rejeita (4 ∤ 6)
├── a=5 → rejeita (5 ∤ 6)
└── a=6 → rejeita (b = 1)
```

Basta um ramo aceitante → 1^6 aceita. Um primo (1^5) fecha **nenhum** ramo.

compostos.jff (Multiple Run) · `python testes/mtnd-compostos/rastreamento.py`

Análise transversal — det × não determinismo

- **MTND $\equiv$ MTD**: por Church-Turing, existe uma MT determinística que reconhece a mesma **L** (simula a árvore por busca). O não determinismo **não aumenta o poder** — só a clareza.
- **SKI é determinístico**: a ordem normal define o próximo passo de forma única; por Church-Rosser, a forma normal (quando existe) é única.
- Dois estilos de "escolha": a MTND **adivinha**; o SKI **reescreve** deterministicamente.

Análise transversal — λ ↔ combinadores

- λ -cálculo, SKI e Máquina de Turing são **equivalentes** (Church-Turing).
- *Bracket abstraction* é uma **tradução construtiva** de um modelo (λ) para outro (SKI)
—
evidência concreta da equivalência.
- **Limites** que aparecem nos dois:
 - SKI: **Problema da Parada** (Ω não normaliza).
 - MTND: L é **decidível**, mas o n° de ramos cresce com $n \rightarrow$ custo (reconhecibilidade $\neq$ custo).

Encerramento

- **Dois modelos, dois problemas distintos**, ambos equivalentes à Máquina de Turing.
- **SKI**: computar sem variáveis — 7 passos no numeral de Church 2; ilustra a Parada.
- **MTND**: adivinhar o fator — aceitação por ramo; 12 estados / 24 transições.
- Tudo **reproduzível**: `test_ski.py` , `demo.py` , `compostos.jff` , `rastreamento.py` .

Obrigado! — Perguntas?